

# Laboratório de Concorrência · Aula 10

André Perin Geraldo · RA 24017529-2 · Turma A

Gerado em 09/10/2026, 20:24:00 · tempo registrado no cronômetro: 0:00

## Preparação: Concorrência em cinco minutos

### 0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A diferença entre cooperação e competição fica bem mais fácil de entender quando olhamos para a nossa rotina. A cooperação funciona como um trabalho em equipe, onde uma etapa depende da outra, igual a um garçom que precisa esperar o cozinheiro terminar o prato para poder levá-lo à mesa. Já a competição é uma disputa por recursos limitados, bem parecido com duas pessoas querendo usar o único banheiro da casa ao mesmo tempo, onde uma precisa esperar a outra sair. No fim das contas, cooperar é conectar etapas e somar forças, enquanto competir é a necessidade de revezar um espaço que não dá para dividir.

## Prática 1: A condição de corrida

### Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Eu esperaria 2.000.000, mas como o contador++ não é atômico, provavelmente saia valor menor

#### 1ª execução (n = 1.000.000)

1893472

#### 2ª execução (n = 1.000.000)

947307

#### 3ª execução (n = 1.000.000)

2000000

### Saída do programa

```
esperado: 2000000
obtido:   1893472
---
esperado: 2000000
obtido:   947307
---
esperado: 2000000
obtido:   2000000
---
(n = 1000)
esperado: 2000
obtido:   2000
esperado: 2000
obtido:   2000
```

### 1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram em duas das três execuções (1893472 e 947307), e os três valores foram diferentes. Mudam porque o escalonador decide a cada execução como intercalar as duas threads; quanto mais vezes as threads leem o mesmo

valor antes de uma delas gravar, mais somas se perdem na 3ª execução o resultado deu certo por acaso, não porque o código esteja correto

### 1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

O contador vale 5 a thread A lê 5, antes de A gravar, a thread B também lê 5 A soma 1 no registrador (6), B soma 1 no registrador (6). A grava 6, depois B grava 6 por cima. Foram feitas duas somas, mas o contador foi de 5 para 6: a atualização de A se perdeu

### 1.3 Com $n = 1000$ , o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com  $n = 1000$  deu 2000 nas duas execuções, ou seja, o erro não apareceu. Cada thread termina tão rápido que quase não há sobreposição entre elas, isso é perigoso porque o programa passa nos testes (com pouca carga) e só falha em produção, com muitos acessos simultâneos e de forma intermitente, difícil de reproduzir e de depurar

## Prática 2: Todas as respostas possíveis (Figura 13.1)

### Ordens testadas no simulador

AAABBB → 8

BBBAAA → 7

ABABAB → 6

AABBAB → 6

AABBBA → 4

### Saída do programa

AAABBB → 8

BBBAAA → 7

ABABAB → 6

Ordens possíveis: 20

TOTAL = 4: 9 ordens, por exemplo AABBBBA

TOTAL = 6: 9 ordens, por exemplo AABABB

TOTAL = 7: 1 ordens, por exemplo BBBAAA

TOTAL = 8: 1 ordens, por exemplo AAABBB

### 2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis e aparecem 4 resultados diferentes (4, 6, 7 e 8) só 1 ordem dá o resultado correto 8 (AAABBB). Ou seja, 18 das 20 ordens dão 4 ou 6, que são resultados com uma operação perdida.

### 2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Ordem AABBBBA: A lê TOTAL = 3; A calcula  $3 + 1 = 4$  no registrador; B lê TOTAL = 3 (A ainda não gravou); B calcula  $3 \times 2 = 6$ ; B grava TOTAL = 6; A grava TOTAL = 4 por cima. A operação de B foi perdida ela foi feita, mas A sobrescreveu o resultado com um valor calculado a partir do TOTAL antigo.

## Prática 3: Corrigindo a corrida

**Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?**

A versão sem sincronização deve ser a mais rápida, mas provavelmente é a errada. As corretas possivelmente são o AtomicInteger e o synchronized

### Saída do programa

```
esperado:          2000000
sem sincronizacao: 1060416 em 11 ms
synchronized:      2000000 em 81 ms
AtomicInteger:      2000000 em 29 ms
---
esperado:          2000000
sem sincronizacao: 2000000 em 6 ms
synchronized:      2000000 em 100 ms
AtomicInteger:      2000000 em 34 ms
```

### 3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Tanto a versão com synchronized quanto a com AtomicInteger funcionaram perfeitamente, batendo os 2.000.000 nas duas rodadas, enquanto a versão sem sincronização mostrou o problema logo de cara ela errou feio na primeira tentativa (ficando em 1.060.416) e deu o resultado certo na segunda por sorte. Claro que essa segurança tem um preço no desempenho, as soluções corretas demoram mais (o synchronized levou de 80 a 100 ms e o AtomicInteger uns 30 ms, contra meros 6 a 11 ms do código livre). Isso acontece porque as threads perdem a total liberdade. No synchronized, elas precisam pedir licença, pegar e soltar a trava do monitor a cada incremento, fazendo uma thread bater no fim da outra. Já o AtomicInteger usa uma instrução atômica direto no processador para sincronizar a memória entre os núcleos. Ele acaba sendo bem mais leve e rápido porque não chega a travar a thread na fila ele apenas tenta atualizar o valor e, se alguém tiver passado na frente, ele refaz a operação rapidinho.

### 3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve, cada saldo ficaria correto isoladamente, mas a transferência são duas operações que precisam acontecer juntas. Entre uma e outra, outra thread poderia ver o dinheiro "sumido" ou fazer outra operação no meio, deixando o sistema inconsistente. É preciso tornar a transferência inteira uma seção crítica travar as duas contas (synchronized/lock, sempre na mesma ordem para evitar impasse) ou usar uma transação no banco de dados.

## Prática 4: Impasse (deadlock)

### Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Acho que nenhuma das mensagens vai aparecer, porque cada thread vai ficar esperando o recurso que a outra está segurando

### Saídas (antes e depois da correção)

```
Antes:
IMPASSE: 2 threads presas

Depois (B pega X e depois Y):
A terminou
B terminou
sem impasse
```

### 4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A pegou X e, depois da pausa, ficou esperando Y. Ao mesmo tempo, B pegou Y e ficou esperando X. Cada uma segura o recurso que a outra precisa e nenhuma solta o que tem, então as duas esperam para sempre

#### 4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Com a mesma ordem, quem pegar X primeiro também consegue pegar Y; a outra thread fica esperando já em X, sem segurar nada assim não existe espera circular A esperando B e B esperando A, que é a condição necessária para o impasse. Exemplo real: um sistema bancário em que uma transferência de A para B trava a conta A e depois a B, enquanto outra transferência de B para A trava B e depois A as duas ficariam presas. A solução é sempre travar as contas em uma ordem fixa por exemplo, pelo número da conta

### Prática 5: Mensagens em vez de memória compartilhada

**Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?**

Com 3 trabalhadores, cerca de 200 ms 2 rodadas de 3 tarefas. Com 1 600 ms, com 6 100 ms.

#### Tempo com 1 trabalhador

600ms

#### Tempo com 3 trabalhadores

200ms

#### Tempo com 6 trabalhadores

100ms

#### Saída do programa

```
trabalhador 3 fez a tarefa 1
trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 2 fez a tarefa 5
trabalhador 3 fez a tarefa 4
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

#### 5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador as 6 tarefas são feitas uma depois da outra  $6 \times 100 = 600$  ms. Com 3, são duas rodadas de 3 tarefas em paralelo: 200 ms. Com 6, todas rodam ao mesmo tempo: 100 ms. Não precisa de synchronized nem de trava porque os trabalhadores não compartilham nenhuma variável: cada tarefa chega por mensagem pelo canal, e o próprio canal garante que cada mensagem é entregue a um único trabalhador. Quem sincroniza é o canal e o WaitGroup para esperar o fim

#### Saídas (antes e depois da troca)

```
Antes:
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
```

```
Depois (make(chan int, 1)):
1
```

## 5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Num canal sem buffer, quem envia fica bloqueado até alguém receber o main envia `ch <- 1` e fica esperando, mas a única goroutine que receberia é o próprio main, que está parado na linha do envio. Ninguém consegue avançar, e o Go detecta o impasse, com buffer de 1, o envio só coloca o valor no espaço do canal e segue depois o main recebe esse valor e imprime 1

## Prática 6: O compilador como guarda

**Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?**

Acho que vai dar erro de compilação, porque o Rust não deixa duas threads mexerem na mesma variável sem uma proteção

**Saídas (o erro e o resultado com move)**

```
error[E0373]: closure may outlive the current function, but it borrows `contador`,
which is owned by the current function
  → p6a_rust_erro.rs:7:27
   |
7 |         let t = thread::spawn(|| {
   |                                ^^ may outlive borrowed value `contador`
8 |             contador += 1;
   |             ----- `contador` is borrowed here
```

Com move:  
warning: value captured by `contador` is never read  
0

**Saída do programa**

```
contador = 400000
```

## 6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread passa a ser dona de uma cópia do número inteiros são do tipo Copy. Ela soma 1 na cópia dela, mas o contador de main continua valendo 0, então é isso que é impresso, o próprio compilador avisa que o valor capturado nunca é lido

## 6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o erro só apareceu em tempo de execução, e nem sempre com  $n = 1000$  passou despercebido em Rust ele foi descoberto em tempo de compilação o programa nem chegou a rodar. A vantagem é que a corrida de dados não chega à produção o custo é que o programador precisa entender as regras de posse e empréstimo e escrever mais código explícito Arc, Mutex, lock, move, e o compilador recusa programas até que essas regras sejam satisfeitas.

## Prática 7: Uma thread só: o laço de eventos

**Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?**

A,D,C,B

## Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

### 7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem foi A, D, C, B. A e D são código síncrono e rodam na hora C é uma microtarefa, que roda assim que o código atual termina, antes de qualquer timer. B, mesmo com 0 ms, é uma macrotarefa ele vai para a fila e só executa depois que o código síncrono e todas as microtarefas terminam

### 7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer estava pronto em 10 ms, mas a única thread estava presa no laço ocupado(300), e o laço de eventos só pega o próximo item da fila quando o código atual termina. Por isso ele rodou depois de 300 ms num servidor Node.js, um cálculo pesado no laço principal trava todas as outras requisições enquanto dura nenhum usuário é atendido. Cálculos pesados devem ir para worker threads, outro processo ou ser divididos em partes menores

## Fechamento: Quem impede a corrida?

---

### Java sem sincronização

Ninguém: há corrida

### Java com synchronized

O programador, com sincronização

### Go com canais

O projeto: não há memória compartilhada

### Rust com Arc e Mutex

O compilador

### JavaScript

Só existe uma thread

### 8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida os dois pedidos leem 1 lugar disponível antes de qualquer um gravar a compra, e os dois compram o mesmo lugar duplica a venda. Para evitar, a verificação e a reserva precisam ser uma operação só uma seção crítica ou, no caso real, uma transação no banco com atualização condicional, por exemplo UPDATE lugares SET dono = WHERE id = AND dono IS NULL, aceitando a compra só se uma linha for alterada.

### 8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que um código mesmo estando errado em java passou certo com n=1000